

Кросплатформне програмування

02

Знайомство з платформою
Java

Історія мови Java

Мова програмування Java була розроблена компанією Sun Microsystems (зараз частина Oracle Corporation) у середині 1990-х років. Головним архітектором мови став Джеймс Гослінг (James Gosling), який очолював команду розробників

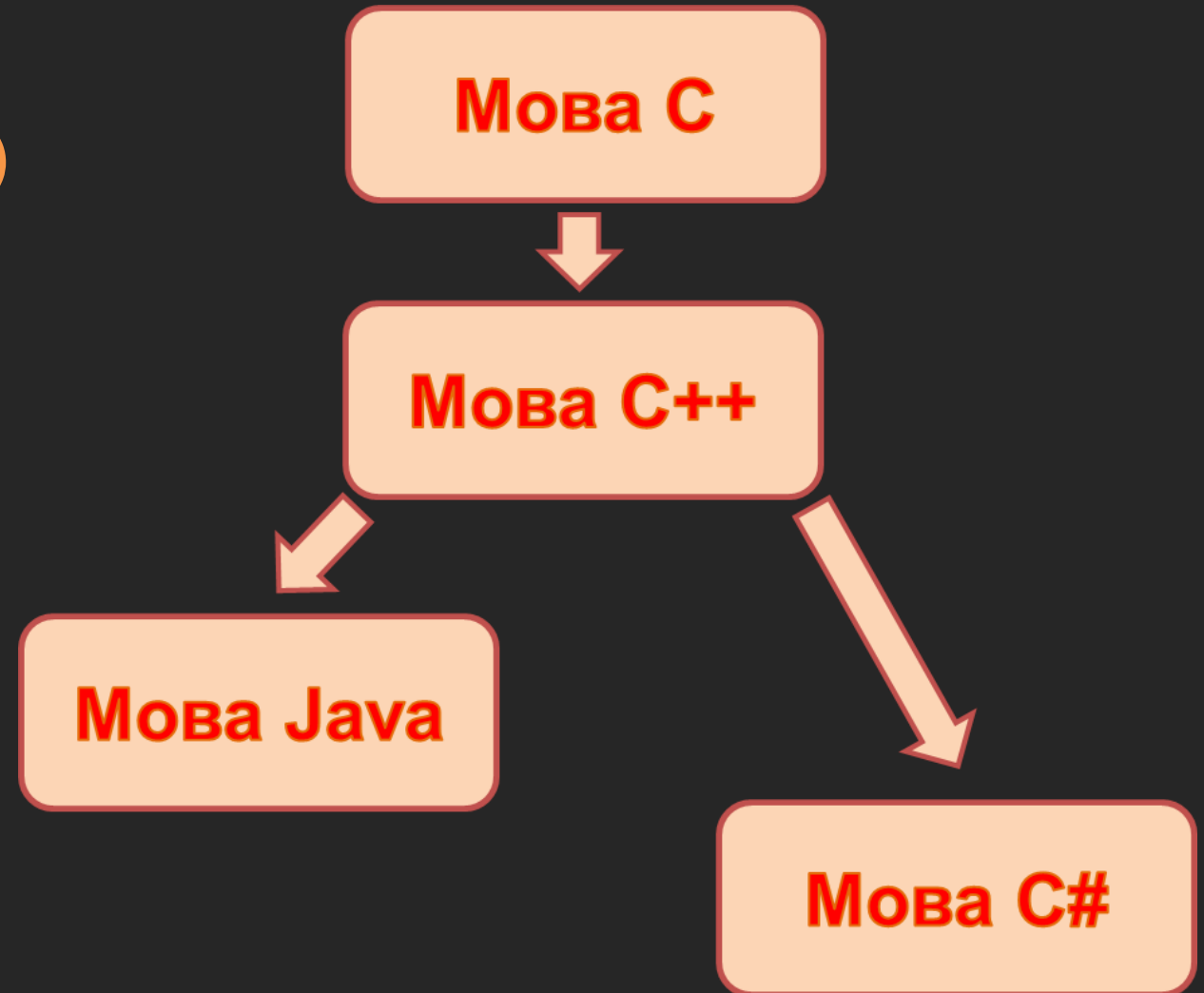
- В середині 1990-х років, в компанії Sun Microsystems виникла необхідність у створенні платформи для програмування домашніх електронних пристроїв (в той час популярні варіанти - мікрохвильові печі, телевізори тощо). Джеймс Гослінг та його команда почали працювати над проектом, який згодом став би мовою Java
- 1991 рік: Перша версія мови отримала назву "Oak". Окрім властивостей для програмування побутових пристроїв, "Oak" мав бути додатково здатний працювати в мережевому середовищі
- 1995 рік: Після запуску World Wide Web та зростання популярності Інтернету, Sun Microsystems перепроєктувала "Oak" на використання в Інтернеті. Основний акцент змістився на можливості розробки динамічних, мережевих додатків. В цьому варіанті мова отримала назву Java
- 23 травня 1995 року: Компанія Sun Microsystems оголосила про випуск мови програмування Java та платформи Java 1.0
- У 2010 році компанія Oracle придбала Sun Microsystems, отримавши контроль над мовою Java. Після цього Java продовжила розвиватися під керівництвом Oracle, і вийшло кілька нових версій з поліпшеннями і новими можливостями



Джеймс Артур Гослінг (1955) — канадський інформатик, найбільш відомий, як засновник та провідний дизайнер мови програмування Java

Генеалогія мови Java

Одна з найпопулярніших
(разом з C++, C# та Python)
мов програмування



Особливості мови Java

Кросплатформність. Одна з найбільших переваг Java - це її кросплатформність. Код, написаний на Java, може виконуватися на будь-якій платформі, на якій є відповідна віртуальна машина Java (JVM). Це означає, що ви можете написати програму один раз і запускати її на різних операційних системах без потреби перекомпіляції

Об'єктно-орієнтованість. Java використовує об'єктно-орієнтований підхід до програмування, що сприяє покращенню структури та організації коду. Всі об'єкти в Java є екземплярами класів, а програми побудовані на основі ієрархії класів

Безпека. Java була розроблена з підвищеною увагою до безпеки. Через використання віртуальної машини Java, контролюється доступ до ресурсів комп'ютера. Також мова має механізми для попередження певних видів програмних помилок, таких як витоки пам'яті та недостовірні вказівники

Автоматичне управління пам'яттю. Java автоматично відслідковує та управляє виділеною пам'яттю для об'єктів. Гарбаж-колектор автоматично видаляє об'єкти, які більше не використовуються, що допомагає уникнути витоків пам'яті

Стандартна бібліотека. Java постачається з великою стандартною бібліотекою, яка містить різні класи та методи для вирішення загальних завдань, таких як робота з рядками, масивами, мережевими операціями тощо

Портативність. Завдяки своїй кросплатформності, Java надає можливість створювати програми, які можуть бути легко перенесені з однієї платформи на іншу без змін вихідного коду

Багатопотоковість. Java має вбудовану підтримку для багатопотокового програмування, що дозволяє створювати програми, які одночасно виконують кілька завдань

Динамічне завантаження класів. Java дозволяє завантажувати класи виключно в момент виконання програми, що дає можливість динамічно розширювати функціональність програми без перекомпіляції

Як компілюється програма в Java

Програма в Java компілюється у спеціальний проміжний формат, який називається байт-код. Байт-код - це низькорівневий набір інструкцій, які можуть виконуватися віртуальною машиною Java (JVM)

Процес компіляції та виконання програми в Java виглядає так:

Створення вихідного коду. Розробник створює вихідний код програми на мові Java. Вихідний код є текстовим файлом з розширенням `.java`

Компіляція. Для того щоб виконати компіляцію вихідного коду, використовується компілятор Java (`javac`). Компілятор перетворює вихідний код у байт-код, який має розширення `.class`. Кожен файл `.class` містить байт-код для відповідного класу з вихідного коду

Виконання на віртуальній машині Java (JVM). Байт-код не може бути виконаний без віртуальної машини Java. Щоб запустити програму, спочатку запускається JVM. JVM інтерпретує байт-код і виконує відповідні інструкції

Зокрема, JVM може виконувати такі дії:

Завантажувати класи, коли вони вперше потрібні програмі

Виконувати оптимізації та управляти роботою пам'яті

Відслідковувати та збирати не використану пам'ять за допомогою гарбж-колектора

Інтерпретація та JIT-компіляція:

JVM може використовувати два підходи до виконання байт-коду: інтерпретація та Just-In-Time (JIT) компіляція

Інтерпретація. В цьому режимі JVM інтерпретує байт-код одну інструкцію за одну, що може бути менш ефективною з точки зору продуктивності

JIT-компіляція. Цей підхід забезпечує вищу продуктивність. JIT-компілятор перетворює байт-код у машинний код для конкретної архітектури процесора в реальному часі під час виконання програми. Це дозволяє досягти швидшого виконання програм

Цей процес дозволяє Java бути кросплатформною: ви пишете код один раз, компілюєте його в байт-код, а потім можна виконувати цей байт-код на будь-якій операційній системі, де є встановлена відповідна віртуальна машина Java

Програмне забезпечення

- **Мінімальна установка:**

JDK та JRE (сайти www.oracle.com та www.java.com)



- **Найпопулярніші IDE:**

IntelliJ IDEA

(сайт www.jetbrains.com)

Eclipse

(сайт www.eclipse.org)

NetBeans

(сайт www.netbeans.apache.org)

Структура програми в Java

- Мова Java є повністю об'єктно-орієнтованою
- Для створення найменшої програми необхідно описати хоча б один клас

Шаблон для створення програми:

```
class ім'я_класу{  
    public static void main(String[] args){  
        // Команди  
    }  
}
```

Програма складається з класу, що містить **головний метод програми**.
Виконання програми – виконання головного методу

Hello Java-World (c)

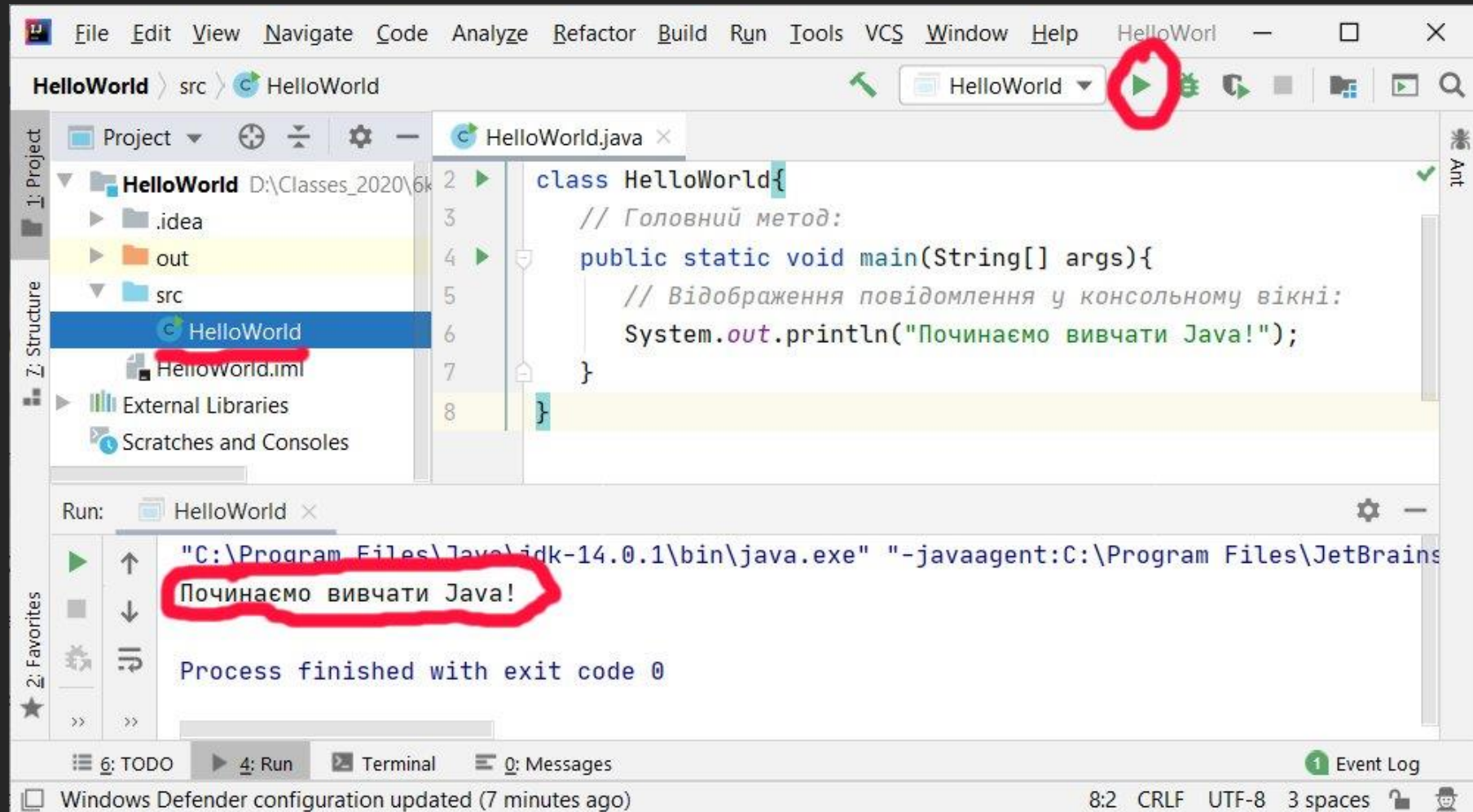
Код програми для виводу текстового повідомлення у консоль:

```
// Головний клас:  
class HelloWorld{  
    // Головний метод:  
    public static void main(String[] args){  
        // Відображення повідомлення у консольному вікні:  
        System.out.println("Починаємо вивчати Java!");  
    }  
}
```

Для виводу повідомлення у консоль використовуємо інструкцію `System.out.println()` або `System.out.print()`

Перша програма: виведення у КОНСОЛЬ

Виконання програми:

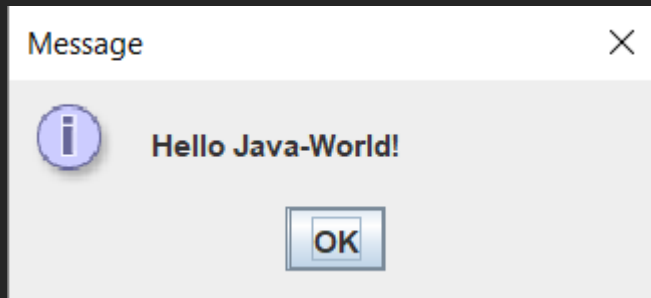


Hello Java-World (c)

Код програми для відображення діалогового вікна:

```
// Інструкція імпорту:  
import javax.swing.JOptionPane;  
class HelloWorld{  
    public static void main(String[] args){  
        // Відображення діалогового вікна:  
        JOptionPane.showMessageDialog(null, "Hello Java-World!");  
    }  
}
```

Результат:

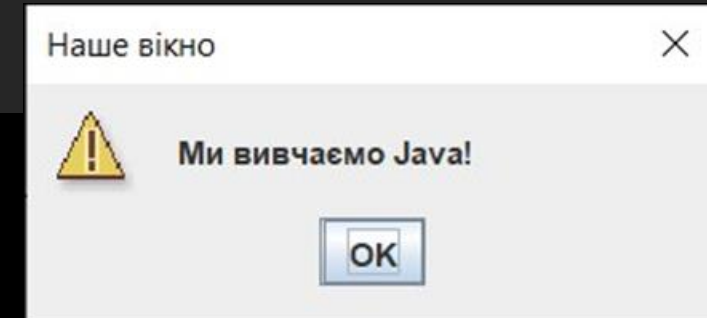




Повідомлення у діалоговому вікні: продовження

Програма для відображення діалогового вікна:

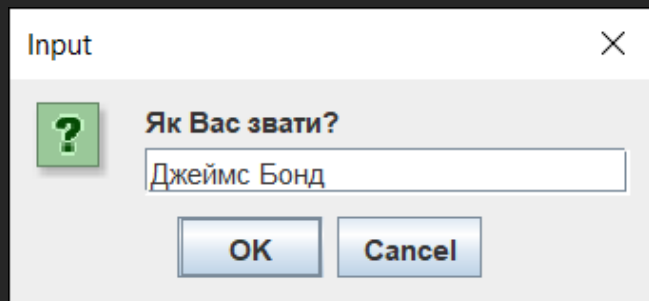
```
// Статичний імпорт:
import static javax.swing.JOptionPane.*;
// Головний клас програми:
class Demo{
    // Головний метод програми:
    public static void main(String[] args){
        // Відображення повідомлення у діалоговому вікні:
        showMessageDialog(null,          // Вікно-контейнер
                          "Ми вивчаємо Java!", // Повідомлення
                          "Наше вікно",      // Заголовок
                          WARNING_MESSAGE     // Піктограма
        );
    }
}
```



Константи для визначення типу піктограми:

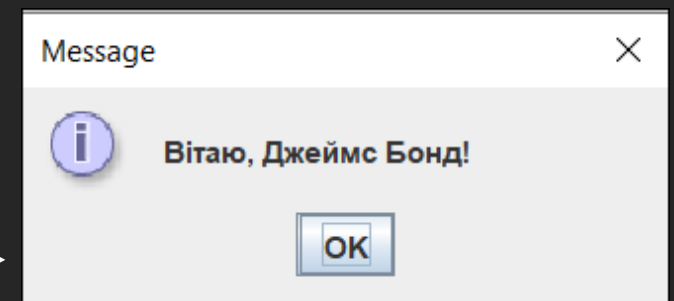
PLAIN_MESSAGE, INFORMATION_MESSAGE, WARNING_MESSAGE, ERROR_MESSAGE,
QUESTION_MESSAGE

```
import static javax.swing.JOptionPane.*;
class Demo{
    public static void main(String[] args){
        // Текст:
        String name;
        // Відображення вікна з полем вводу:
        name=showInputDialog("Як Вас звати?");
        // Відображення вікна з повідомленням:
        showMessageDialog(null,"Вітаю, "+name+"!");
    }
}
```



**У першому вікні
вводиться ім'я**

**Ім'я з'являється у
другому вікні**





Програма: зчитування числа

```
import static javax.swing.JOptionPane.*;
class Demo{
    public static void main(String[] args){
        // Цілочислові змінні:
        int now=2023,age,year;
        // Текст:
        String res,txt="Ваш вік: ";
        // Відображення вікна з полем вводу:
        res=showInputDialog(null,                // Вікно-контейнер
                            "У якому році Ви народились?", // Текст
                            "Визначаємо вік",           // Назва
                            QUESTION_MESSAGE            // Піктограма
                            );
        // Перетворення тексту на число:
        year=Integer.parseInt(res);
        // Розрахунок віку:
        age=now-year;
        // Відображення вікна з повідомленням:
        showMessageDialog(null,txt+age+" років");
    }
}
```



Програма: зчитування числа

The screenshot shows the IntelliJ IDEA IDE with a project named 'Demo'. The 'src' directory contains a file 'Demo.java'. The code in 'Demo.java' is as follows:

```
1 import static javax.swing.JOptionPane.*;
2 class Demo{
3     public static void main(String[] args){
4         // Цілочислові змінні:
5         int now=2023,age,year;
6         // Текст:
7         String res,txt="Ваш вік: ";
8         // Відображення вікна з полем вводу:
9         res=showInputDialog(null,                // Вікно-контейнер
10             "У якому році Ви народились?", // Текст
11             "Визначаємо вік",              // Назва
12             QUESTION_MESSAGE               // Піктограма
13         );
14         // Перетворення тексту на число:
15         year=Integer.parseInt(res);
16         // Розрахунок віку:
17         age=now-year;
18         // Відображення вікна з повідомленням:
19         showMessageDialog(null,txt+age+" років");
20     }
21 }
```

In the foreground, a dialog box titled 'Визначаємо вік' (Determining age) is open. It contains a question mark icon and the text 'У якому році Ви народились?' (In which year were you born?). The input field contains the value '1812'. There are 'OK' and 'Cancel' buttons.

З'являється після
закриття першого вікна



The screenshot shows a 'Message' dialog box with an information icon and the text 'Ваш вік: 211 років' (Your age: 211 years). There is an 'OK' button.



Програма: введення з консолі

```
// Імпорт класу Scanner:
import java.util.Scanner;
class Demo{
    public static void main(String[] args){
        // Створення об'єкту класу Scanner:
        Scanner scan=new Scanner(System.in);
        // Оголошення змінних:
        int x,y;
        String txt;
        System.out.print("Уведіть перше число: ");
        // Зчитування першого числа:
        x=Integer.parseInt(scan.nextLine());
        System.out.print("Уведіть текст: ");
        // Зчитування тексту:
        txt=scan.nextLine();
        System.out.print("Уведіть друге число: ");
        // Зчитування другого числа:
        y=scan.nextInt();
        // Відображення зчитаних значень:
        System.out.println("Ви ввели числа: "+x+" и "+y);
        System.out.println("Ви ввели текст: "+txt);
    }
}
```



Програма: введення з консолі

The screenshot displays the IntelliJ IDEA IDE with a project named 'Demo'. The 'src' directory contains a file 'Demo.java'. The code in 'Demo.java' is as follows:

```
1 // Імпорт класу Scanner:
2 import java.util.Scanner;
3 class Demo{
4     public static void main(String[] args){
5         // Створення об'єкту класу Scanner:
6         Scanner scan=new Scanner(System.in);
7         // Оголошення змінних:
8         int x,y;
9         String txt;
10        System.out.print("Уведіть перше число: ");
11        // Зчитування першого числа:
12        x=Integer.parseInt(scan.nextLine());
13        System.out.print("Уведіть текст: ");
14        // Зчитування тексту:
15        txt=scan.nextLine();
16        System.out.print("Уведіть друге число: ");
17        // Зчитування другого числа:
18        y=scan.nextInt();
19        // Відображення зчитаних значень:
20        System.out.println("Ви ввели числа: "+x+" и "+y);
```

The 'Run' window at the bottom shows the execution of the program. The command used is: `"C:\Program Files\Java\jdk-15.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2020"`. The output is as follows:

```
Уведіть перше число: 123
Уведіть текст: Вивчаємо Java
Уведіть друге число: 321
Ви ввели числа: 123 и 321
Ви ввели текст: Вивчаємо Java

Process finished with exit code 0
```

The status bar at the bottom indicates: `Build completed successfully in 4 s 174 ms (a minute ago)`.

Вікно з картинкою

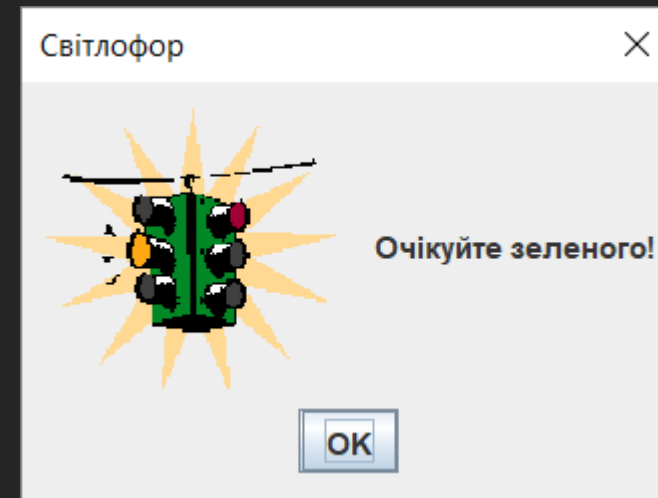
```
// Імпорт бібліотеки:
import javax.swing.*;
// Статичний імпорт з класу JOptionPane:
import static javax.swing.JOptionPane.*;
class Demo{
    public static void main(String[] args){
        // Відображення вікна з картинкою:
        showMessageDialog(null,
            // Об'єкт зображення:
            new ImageIcon("D:/Pictograms/panda.jpg") ,
            "Панда",          // Назва
            PLAIN_MESSAGE     // Немає піктограми
        );
    }
}
```



Файл panda.jpg має знаходитись у папці D:/Pictograms/

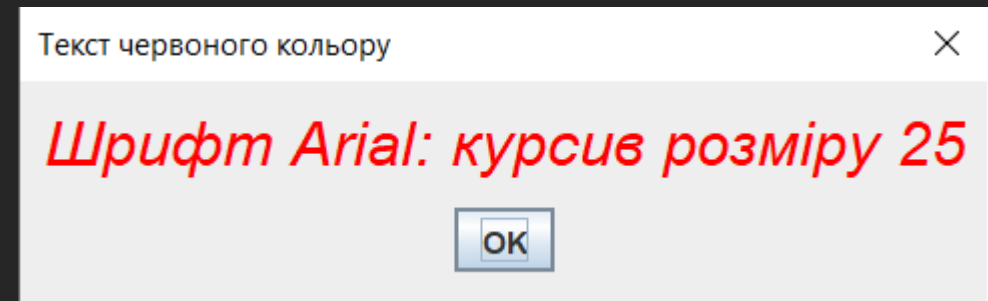
Вікно з користувацькою піктограмою

```
import javax.swing.*;
class Demo{
    public static void main(String[] args){
        // Піктограма для нового вікна:
        ImageIcon icon=new ImageIcon("/pictograms/my pict.png");
        JOptionPane.showMessageDialog(null,
            "Очікуйте зеленого!", // Повідомлення
            "Світлофор", // Заголовок
            JOptionPane.PLAIN_MESSAGE, // Аргумент ігнорується
            icon // Піктограма
        );
    }
}
```



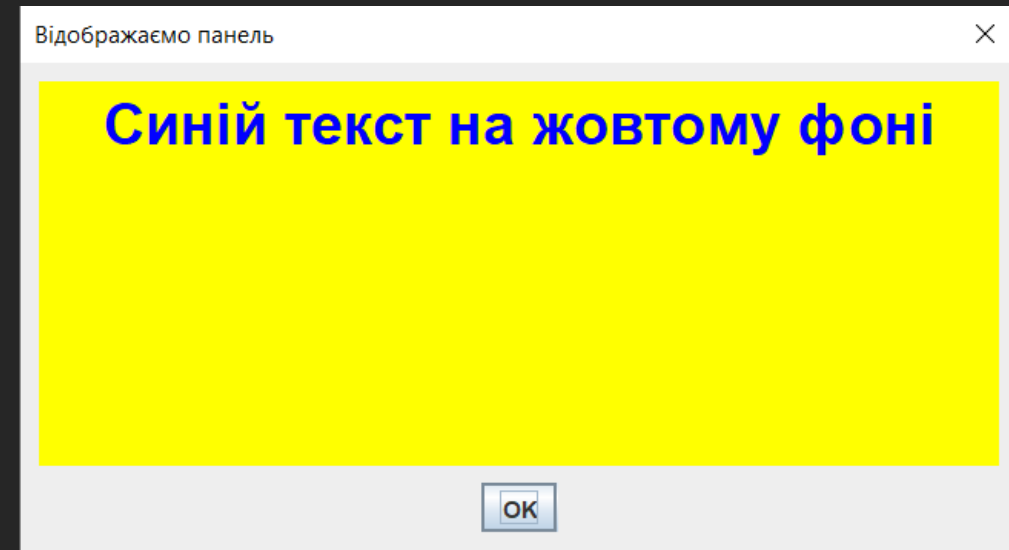
Вікно з кольоровим текстом

```
import java.awt.*;
import javax.swing.*;
class Demo{
    public static void main(String[] args){
        // Створюємо мітку з текстом:
        JLabel lbl=new JLabel("Шрифт Arial: курсив розміру 25");
        // Задаємо колір для тексту мітки (червоний):
        lbl.setForeground(Color.RED);
        // Створюємо шрифт для тексту мітки:
        Font F=new Font("Arial",Font.ITALIC,25);
        // Застосовуємо шрифт до мітки:
        lbl.setFont(F);
        // Відображаємо діалогове вікно:
        JOptionPane.showMessageDialog(null, // Батьківське вікно
            lbl, // Мітка для відображення у вікні
            "Текст червоного кольору", // Заголовок вікна
            JOptionPane.PLAIN_MESSAGE // Тип повідомлення
        );
    }
}
```



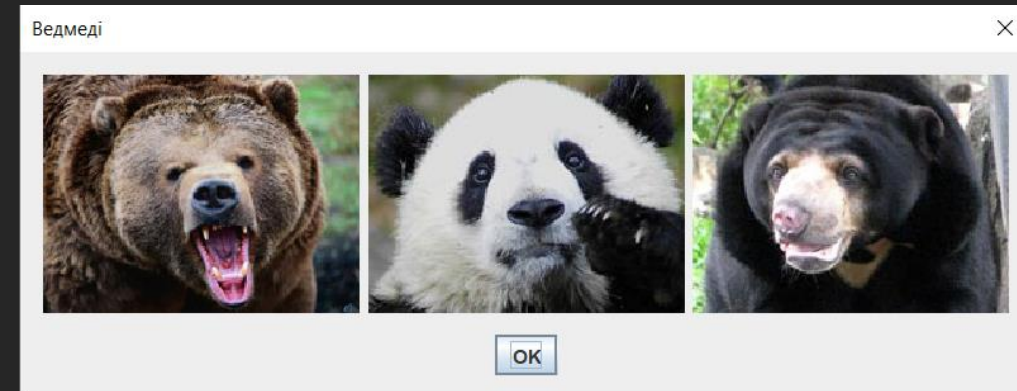
Кольоровий фон у вікні з повідомленням

```
import java.awt.*;
import javax.swing.*;
class Demo{
    public static void main(String[] args){
        // Створюємо об'єкт панелі:
        JPanel pnl=new JPanel();
        // Встановлюємо розміри панелі:
        pnl.setPreferredSize(new Dimension(500,200));
        // Задаємо колір тла для панелі:
        pnl.setBackground(Color.YELLOW);
        // Створюємо мітку з текстом:
        JLabel lbl=new JLabel("Синій текст на жовтому фоні");
        // Задаємо колір тексту для мітки:
        lbl.setForeground(Color.BLUE);
        // Застосовуємо шрифт для мітки:
        lbl.setFont(new Font("Arial",Font.BOLD,30));
        // Додаємо мітку на панель:
        pnl.add(lbl);
        // У вікні повідомлення відображається панель:
        JOptionPane.showMessageDialog(null, // Батьківське вікно
            pnl, // Посилання на панель
            "Відображаємо панель", // Заголовок вікна
            JOptionPane.PLAIN_MESSAGE // Тип вікна
        );
    }
}
```



Декілька зображень у вікні

```
import java.awt.*;
import javax.swing.*;
class Demo{
    public static void main(String[] args){
        // Створюємо панель:
        JPanel pnl=new JPanel();
        // Допоміжна текстова змінна
        // (Каталог із файлами зображень):
        String path="/pictograms/";
        // Назви файлів із зображеннями:
        String[] files={"grizli.jpg","panda.jpg","malai.jpg"};
        // Масив міток (масив об'єктних посилань):
        JLabel[] lbl=new JLabel[files.length];
        // Створюємо мітки та додаємо їх на панель:
        for(int i=0;i<lbl.length;i++){
            // Створюємо мітку із піктограмою:
            lbl[i]=new JLabel(new ImageIcon(path+files[i]));
            // Додаємо мітку із зображенням на панель:
            pnl.add(lbl[i]);
        }
        // Відображаємо діалогове вікно:
        JOptionPane.showMessageDialog(null, // Батьківське вікно
            pnl, // Посилання на панель
            "Ведмеді", // Заголовок вікна
            JOptionPane.PLAIN_MESSAGE // Тип вікна
        );
    }
}
```



Дякую за увагу!

Запитання?